Bits, Bytes, and Integers

CSE 238/2038/2138: Systems Programming LAB

Instructor:
Fatma CORUT ERGİN

Teaching Assistants:
Lokman ALTIN – Zuhal ALTUNTAŞ

Integer Representation
Unsigned Integers

- An unsigned number represents positive numbers between

  0 and **2^k-1 (U_{max})**, where *k* is the number of bits used.

  - k=5 -> $(00000)_2$ … $(11111)_2$ -> 0 … 2^5-1 -> 0 … 31 -> 0 … U_{max}
- Subtracting 1 from 0 will *underflow* to the highest value (U_{max})
  - $(00000)_2 - (00001)_2 = (11111)_2$
- Adding 1 to the highest value (U_{max}) will *overflow* to 0
  - $(11111)_2 + (00001)_2 = (00000)_2$

Integer Representation
Unsigned Integers

- 8-bit unsigned integers:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
1	0	1	1	0	0	1	0	**= 178**

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
0	0	1	0	1	0	1	0	**= 42**

Integer Representation
Signed Integers

- Two's Complement Representation used
- A signed number represents numbers between **-2^{k-1} (T_{min})** and **$-2^{k-1}-1$ (T_{max})**, where *k* is the number of bits used.
  - k=5 -> $(10000)_2$ … $(01111)_2$ -> -2^{5-1} … $2^{5-1}-1$ -> -16 … 15 -> T_{min} … T_{max}
- Subtracting 1 from the smallest value will *underflow* to the highest value (T_{max})
  - $(10000)_2 - (00001)_2 = (01111)_2$
- Adding 1 to the highest value (T_{max}) will *overflow* to the smallest value (T_{min})
  - $(01111)_2 + (00001)_2 = (10000)_2$

Integer Representation
Signed Integers

- 8-bit signed integers:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
1	0	1	1	0	0	1	0	**= -78**

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
0	0	1	0	1	0	1	0	**= 42**

Sizes for Different Data Types in C

Let's compile this code (dataTypes.c)

```
#include <stdio.h>
#include <limits.h>

int main(){
    printf("unsigned short max: %hu = 0x%X\n",USHRT_MAX, USHRT_MAX);
    printf("unsigned max: %u = 0x%X\n",UINT_MAX, UINT_MAX);
    printf("unsigned long max: %lu = 0x%X\n\n",ULONG_MAX, ULONG_MAX);


    printf("signed short min: %d = 0x%X\n",SHRT_MIN, SHRT_MIN);
    printf("signed short max: %d = 0x%X\n\n",SHRT_MAX, SHRT_MAX);

    printf("signed int min: %d = 0x%X\n",INT_MIN, INT_MIN);
    printf("signed int max: %d = 0x%X\n\n",INT_MAX, INT_MAX);

    return 0;
}
```

Conversion Between Signed & Unsigned

Let's compile this code (typeConversion.c)

```
#include <stdio.h>

int main(){
    short int x = -15;
    unsigned short ux = (unsigned short) x; //explicit casting
    printf("x = %d (0x%X), ux = %hu (0x%x)\n", x, x, ux, ux);

    unsigned short u = 65535u; //Umax
    short tu = u;   //implicit casting
    printf("u = %hu (0x%X), tu = %d (0x%X)\n", u,u,tu,tu);

    return 0;
}
```

Sign Extension Example (in C)

Let's compile this code (signExtension.c)

```
#include <stdio.h>

int main(){

    short sx =  -12345;
    unsigned ux = sx;    /* ux = 53191 */
    unsigned uy = (unsigned)(unsigned short) sx; /* uy = 53191 */

    printf(" sx=%hi (0x%x)\n ux=%hu (0x%.8x)\n uy=%d
       (0x%.8x)\n\n",sx,sx,ux,ux,uy,uy);

return 0;
}
```

Truncation Example (in C)

Let's compile this code (truncate.c)

```
#include <stdio.h>

int main(){

    int x = 53191;
    short sx = (short int) x;  /* sx = -12345 */
    int y = sx;      /*  y = -12345 */

    printf(" x=%d (0x%.8x)\n sx=%hi (0x%.4x)\n y=%d
                (0x%.8x)",x,x,sx,sx,y,y);

    return 0;
}
```

Logical Operations

Let's compile this code (logOps.c)

```
#include <stdio.h>

int main(){

    int a = 12;
    printf("%d && 0 = %d\n",a,a&&0);
    printf("%d && 1 = %d\n\n",a,a&&1);

    printf("%d || 0 = %d\n",a,a||0);
    printf("%d || 1 = %d\n\n",a,a||1);

    printf("%d == 0 = %d\n",a,a==0);
    printf("%d == 1 = %d\n\n",a,a==1);

    printf("!%d = %d\n",a,!a);

    return 0;
}
```

Bitwise Logical Operations

Let's compile this code (bitOps.c)

```
#include <stdio.h>

int main(){

    short a = 13;
    printf("%d & 0 = %d (%.4X & %.4X = %.4X)\n",a,a&0,a,0,a&0);
    printf("%d & 1 = %d (%.4X & %.4X = %.4X)\n\n",a,a&1,a,1,a&1);

    printf("%d | 0 = %d (%.4X | %.4X = %.4X)\n",a,a|0,a,0,a|0);
    printf("%d | 1 = %d (%.4X | %.4X = %.4X)\n\n",a,a|1,a,1,a|1);

    printf("%d ^ 0 = %d (%.4X ^ %.4X = %.4X)\n",a,a^0,a,0,a^0);
    printf("%d ^ 1 = %d (%.4X ^ %.4X = %.4X)\n\n",a,a^1,a,1,a^1);

    printf("~%d = %d (~%.4X = %.4X)\n",a,~a,a,~a);

    return 0;
}
```

Bitwise Logical Operations

Let's compile this code(bitOps_v2.c)

```
#include <stdio.h>

int main(){

    unsigned short a = 13;
    printf("%hu & 0 = %hu (%.4X & %.4X = %.4X)\n",a,a&0,a,0,a&0);
    printf("%hu & 1 = %hu (%.4X & %.4X = %.4X)\n\n",a,a&1,a,1,a&1);

    printf("%hu | 0 = %hu (%.4X | %.4X = %.4X)\n",a,a|0,a,0,a|0);
    printf("%hu | 1 = %hu (%.4X | %.4X = %.4X)\n\n",a,a|1,a,1,a|1);

    printf("%hu ^ 0 = %hu (%.4X ^ %.4X = %.4X)\n",a,a^0,a,0,a^0);
    printf("%hu ^ 1 = %hu (%.4X ^ %.4X = %.4X)\n\n",a,a^1,a,1,a^1);

    printf("~%hu = %hu (~%.4X = %.4X)\n",a,~a,a,~a);

    return 0;
}
```

Power-of-2 Multiply with Shift (in C)

Let's compile this code(leftShift.c)

```
#include <stdio.h>

int main(){

  int x=5; /* 00..0101 in binary, 00000005 in hex */
  int y = x<<4; /* x<<4 = 00..01010000 in binary, 00000050 in hex */
  printf("%d<<4 = %d*(2^4) = %d (0x%.8X<<4 = 0x%.8X)\n", x,x,y,x,y);

  return 0;
}
```

Unsigned Power-of-2 Divide with Shift (in C)

Let's compile this code(rightShift.c)

```
#include <stdio.h>

int main(){

    unsigned x=128; /* 0000 0000 1000 0000 in binary, 0080 in hex */
    int y = x>>4;
    /* x>>4 = 00..0001000 in binary, 0008 in hex */
    printf("%d>>4 = %d/(2^4) = %d (0x%.8X>>4 = 0x%.8X)\n", x,x,y,x,y);

    return 0;
}
```

Unsigned Power-of-2 Divide with Shift (in C)

Let's compile this code(rightShift.c)

```
#include <stdio.h>

int main(){

    unsigned x=128; /* 0000 0000 1000 0000 in binary, 0080 in hex */
    int y = x>>4;
    /* x>>4 = 00..0001000 in binary, 0008 in hex */
    printf("%d>>4 = %d/(2^4) = %d (0x%.8X>>4 = 0x%.8X)\n", x,x,y,x,y);

    return 0;
}
```

Also, test with "x=127" and "x=129"

Signed Power-of-2 Divide with Shift (in C)

Let's compile this code(signedRightShift.c)

```
#include <stdio.h>

int main(){

    int x=-16; /* 1111 1111 1111 0000 in binary, FFF0 in hex */
    int y = x>>4;
    /* x>>4 = 11..11 in binary, FFFF in hex */
    printf("%d>>4 = %d/(2^4) = %d (0x%.8X>>4 = 0x%.8X)\n", x,x,y,x,y);

    return 0;
}
```

Signed Power-of-2 Divide with Shift (in C)

Let's compile this code(signedRightShift.c)

```
#include <stdio.h>

int main(){

    int x=-16; /* 1111 1111 1111 0000 in binary, FFF0 in hex */
    int y = x>>4;
    /* x>>4 = 11..11 in binary, FFFF in hex */
    printf("%d>>4 = %d/(2^4) = %d (0x%.8X>>4 = 0x%.8X)\n", x,x,y,x,y);

    return 0;
}
```

Also, test with "x=-15" and "x=-17"

Little Endian Representation

Let’s compile this code(littleEndian.c)

```
#include <stdio.h>

typedef unsigned char *ucpointer;

void show_bytes(ucpointer start, size_t len){
  size_t i;
  for (i=0; i<len; i++)
    printf("%p\t0x%.2x\n",start+i, start[i]);
}

int main(){
    int a = 12345;
    printf("int a = 12345;\n");
    show_bytes((ucpointer) &a, sizeof(int));

    return 0;
}
```

Little Endian Representation

Let's compile this code(littleEndian.c)

```
#include <stdio.h>

typedef unsigned char *ucpointer;

void show_bytes(ucpointer start, size_t len){
  size_t i;
  for (i=0; i<len; i++)
    printf("%p\t0x%.2x\n",start+i, start[i]);
}

int main(){
    int a = 12345;
    printf("int a = 12345;\n");
    show_bytes((ucpointer) &a, sizeof(int));

    return 0;
}
```

Check for the result in different runs & different machines

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	True
`x <= 0 ⇒ -x >= 0`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	True
`x <= 0 ⇒ -x >= 0`	False
`(x|-x)>>31 == -1`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	True
`x <= 0 ⇒ -x >= 0`	False
`(x|-x)>>31 == -1`	False
`ux >> 3 == ux/8`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	True
`x <= 0 ⇒ -x >= 0`	False
`(x|-x)>>31 == -1`	False
`ux >> 3 == ux/8`	True
`x >> 3 == x/8`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	True
`x <= 0 ⇒ -x >= 0`	False
`(x|-x)>>31 == -1`	False
`ux >> 3 == ux/8`	True
`x >> 3 == x/8`	False
`x & (x-1) != 0`	

Integer C Puzzles

Initialization

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

`(x < 0) ⇒ ((x*2) < 0)`	False
`ux >= 0`	True
`x & 7 == 7 ⇒ (x<<30) < 0`	True
`ux > -1`	False
`x > y ⇒ -x < -y`	False
`(x*x) >= 0`	False
`(x>0) && y > 0 ⇒ x+y > 0`	False
`x >= 0 ⇒ -x <= 0`	True
`x <= 0 ⇒ -x >= 0`	False
`(x|-x)>>31 == -1`	False
`ux >> 3 == ux/8`	True
`x >> 3 == x/8`	False
`x & (x-1) != 0`	False